

Article

Model Predictive Control of Quadruped Robot Based on Reinforcement Learning

Zhitong Zhang [†] , Xu Chang [†], Hongxu Ma, Honglei An and Lin Lang ^{*}College of Intelligence Science and Technology, National University of Defense Technology,
Changsha 410073, China^{*} Correspondence: langlin_8502@nudt.edu.cn[†] These authors contributed equally to this work.

Abstract: For the locomotion control of a legged robot, both model predictive control (MPC) and reinforcement learning (RL) demonstrate powerful capabilities. MPC transfers the high-level task to the lower-level joint control based on the understanding of the robot and environment, model-free RL learns how to work through trial and error, and has the ability to evolve based on historical data. In this work, we proposed a novel framework to integrate the advantages of MPC and RL, we learned a policy for automatically choosing parameters for MPC. Unlike the end-to-end RL applications for control, our method does not need massive sampling data for training. Compared with the fixed parameters MPC, the learned MPC exhibits better locomotion performance and stability. The presented framework provides a new choice for improving the performance of traditional control.

Keywords: model predictive control; reinforcement learning; parameter adaptive; quadruped robot



Citation: Zhang, Z.; Chang, X.; Ma, H.; An, H.; Lang, L. Model Predictive Control of Quadruped Robot Based on Reinforcement Learning. *Appl. Sci.* **2023**, *13*, 154. <https://doi.org/10.3390/app13010154>

Academic Editors: Yue Wu,
Xinglong Zhang and Pengfei Jia

Received: 8 November 2022

Revised: 16 December 2022

Accepted: 20 December 2022

Published: 22 December 2022



Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The quadrupeds are outstanding in high-speed running, weight-bearing, and terrain passing. For example, cheetahs' sprints have been measured at a maximum of 114 km/h, and they routinely reach velocities of 80–100 km/h while pursuing prey. Goliath frogs can jump up to 5 m high while their body length is only from 17 to 32 cm. Blue sheep are usually found near cliffs, in preparation to run toward rugged slopes to avoid danger. Camels are known as the “ships of the desert”, they are able to carry hundreds of kilos of cargo for long trips in harsh desert environments. These striking features have inspired researchers' enthusiasm for the bionic quadruped robots. It is hoped that one day such quadruped robots can surpass animals in movement skills and perform tasks in challenging environments. These visions put forward higher requirements for the performance of robot controllers.

Half a century has witnessed the development of legged robots, and many excellent quadruped robots have emerged. The issue of controller design has shifted from static position planning [1] to highly dynamic optimization [2,3]. Among numerous optimization algorithms, Model Predictive Control (MPC) has emerged as the most widely used control algorithm in the robot field.

There are three main elements of MPC [4]: the predictive model, the reference trajectory, and the control algorithm. This is now more clearly stated as model-based prediction, receding horizontal optimization, and feedback correction. The vast literature invariably says that the greatest attraction of predictive control is its ability to deal explicitly with control and state quantity constraints, this ability arises from the predictions of future dynamic behavior based on the analytical model of the control object, by adding the constraints to the future inputs, outputs, and state. The receding horizontal optimization ensures that the system can quickly respond to the uncertainty from the internal system or external environment.

Since MIT Biomimetic Robotics Lab has open-sourced Cheetah-Software [5], MPC has become a new baseline method for the locomotion control of quadruped robots. Bledt proposed the policy-regularized MPC [6] to stabilize different gaits with several heuristic reference policies. Ding presented the representation-free MPC [7] framework that directly represents orientation using the rotation matrix and can stabilize dynamic motions that involve the singularity. Liu proposed a design approach of gait parameters with minimum energy consumption [8]. Chang presented the Proportional Differential MPC (PDMPC) controller [9] that has the ability to compensate for the unmodeled leg mass or payload. Although MPC has made significant progress in the field of legged-robot control, it still faces many challenges. The precise predictive model is difficult to establish; the simplified model will introduce model mismatch, and the fixed parameters controller does not have strong generalization ability.

In recent years, with the revival of artificial intelligence, achieving autonomous learning control for robots has become one of the research highlights. Kolter [10] learned parameters through policy search based on fixed strategies and realized the action of jumping from the ground to obstacles on Boston Dynamics' little dog. Tuomas Haarnoja [11] developed a variant of the soft actor-critic algorithm to realize the level walking of a quadruped robot with only 8 Degree-of-Freedom (DOF). Yao proposed a video imitation adaptation network [12] that can imitate the action of animals and adapt it to the robot from a few seconds of video. Hwangbo applied model-free reinforcement learning on ANYmal, realized omnidirectional motion on flat ground, and fall recovery. The trained controller can follow the command speed well in any direction. In order to enable the simulation results to be transferred to the real robots, domain randomization, actuator modeling, and random noise during training were used [13]. Additionally, the two-stage training process makes the student policy reconstruct the latent information which not directly observable, such as contact states, terrain shape, and external disturbances. Without vision and environmental information, the robot successfully passed through various complex terrains only with proprioceptive information [14]. Training of reinforcement learning requires a large number of samples, the end-to-end controller policy lack of interpretability, the curse of dimensionality, and the curse of goal specification challenge the usage of reinforcement learning [15]. More importantly, conventional model-based control methods concentrate the intelligence of human researchers, and cannot be discarded roughly.

This work is an extension of our previous work [9]. In this work, we proposed a novel approach to combine the advantages of model predictive control and reinforcement learning. PDMPC with a dozen parameters is considered a parametric controller that provides stable control to generate samples, reinforcement learning training the policy networks to modify parameters online. Compared with the fixed parameters controller, the learned controller has better performance in command tracking and equilibrium stability.

The rest of this paper is organized as follows. Section 2 briefly presents the MPC with Proportional Differential (PD) compensator. Section 3 presents the details of the reinforcement learning framework for PDMPC. The simulation setup and results are illustrated in Section 4. Finally, Section 5 concludes this paper.

2. PDMPC Formulation

Our quadruped robot is Unitree A1, as shown in Figure 1. It has four elbow-like legs, and each leg has three degrees of freedom, called hip side, hip front and knee respectively according to the order of attachment. The first two joints are directly driven, and the knee joint is driven by a connecting rod. This design concentrates the majority of quality on the body.



Figure 1. Unitree A1 robot.

In order to reduce the computational consumption and the difficulty of optimization solution, the dynamic model of the quadruped robot used as a predictive model is simplified to a single rigid body model with four massless variable-length rods:

$$\ddot{\mathbf{p}} = \frac{\sum_{i=1}^k \mathbf{f}_i}{m} + \mathbf{g} \quad (1)$$

$$\frac{d}{dt}(\mathbf{I}\boldsymbol{\omega}) = \mathbf{I}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega} = \sum_{i=1}^k (\mathbf{r}_i \times \mathbf{f}_i) \quad (2)$$

where m and $\mathbf{I}$ are the mass and body inertia matrix, $\mathbf{p}$ and $\boldsymbol{\omega}$ are the positions and angular velocity of the body, $\mathbf{g}$ is the gravitational acceleration vector, $\mathbf{f}_i$ and $\mathbf{r}_i$ are the foot reaction force and the position vector from the foot end to the center of mass (CoM) in the world coordinate system. $\boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega}$ is neglected under the assumption that small angular velocity during the robot locomotion.

The control framework of MPC is shown in Figure 2. The estimated states of the quadruped robot are 6-DOF pose and corresponding 6-DOF velocity, the inputs are gait pattern, desired speed, and attitude angle, the reference trajectory generator plans a desired path within the prediction horizon based on the user inputs, and the current state, the swing planner schedules the legs' phase and the foot trajectory, the MPC controller outputs the desired ground reaction force $\mathbf{f}_i^0$, at last, the low-level joint torque/position controller executes the control commands to drive the robot.

$$\boldsymbol{\tau}_i = \mathbf{J}_i^T \mathbf{R}^T \mathbf{f}_i^0 + \mathbf{k}_p (\mathbf{q}^{des} - \mathbf{q}) + \mathbf{k}_d (\dot{\mathbf{q}}^{des} - \dot{\mathbf{q}}) \quad (3)$$

where $\boldsymbol{\tau}$ is the joint torque vector, $\mathbf{R}$ is the rotation matrix of body, the leg jacobian matrix $\mathbf{J}$ maps the force in operation space to the torque in joint space. $\mathbf{q}^{des}$ and $\dot{\mathbf{q}}^{des}$ are the desired trajectory of joint, $\mathbf{k}_p$ and $\mathbf{k}_d$ are the gain of the PD controller.

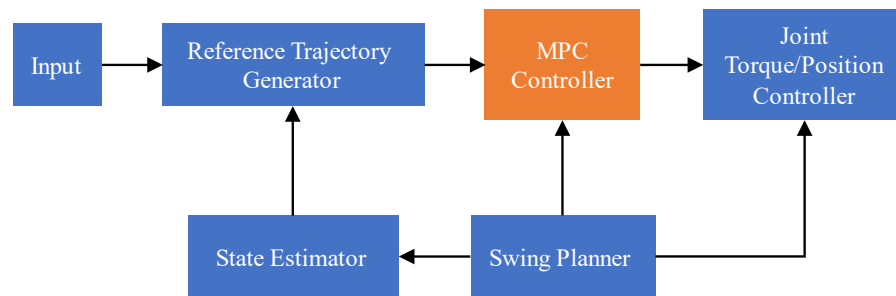


Figure 2. Control framework.

Due to the above simplification, model mismatch will inevitably occur, especially when the leg mass is large or the dynamic parameter estimation is inaccurate, and the

conventional MPC is no longer effective to the uncertainty. Therefore, PDMPC is designed to solve this problem. The structure of the PDMPC controller is shown in Figure 3.

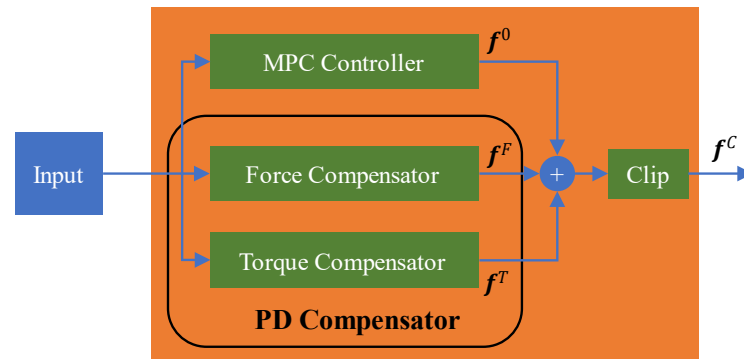


Figure 3. PDMPC architecture.

Based on the above-simplified model in Equations (1) and (2), the MPC controller obtains the expected force f^0 of the stance leg through linear optimization. As well, our PD compensator is proposed to compensate for the model uncertainty. We divide the compensator into two parts based on force and torque.

$$F_e = K_f(p_{des} - p) + D_f(\dot{p}_{des} - \dot{p}) \quad (4)$$

$$\tau_e = K_\tau(\Theta_{des} - \Theta) + D_\tau(\omega_{des} - \omega) \quad (5)$$

where Θ is the Euler angle, F_v and τ_v are the additional virtual force and torque acting on the body. K and D are the diagonal gain matrix with corresponding dimensions.

$$K_f = \begin{bmatrix} k_{fx} & & \\ & k_{fy} & \\ & & k_{fz} \end{bmatrix} \quad D_f = \begin{bmatrix} d_{fx} & & \\ & d_{fy} & \\ & & d_{fz} \end{bmatrix} \quad (6)$$

$$K_\tau = \begin{bmatrix} k_{\tau\varphi} & & \\ & k_{\tau\theta} & \\ & & k_{\tau\psi} \end{bmatrix} \quad D_\tau = \begin{bmatrix} d_{\tau\varphi} & & \\ & d_{\tau\theta} & \\ & & d_{\tau\psi} \end{bmatrix} \quad (7)$$

The force compensator is used to strengthen the tracking of linear motion. In order to reasonably distribute the virtual force to each stance leg, we describe this problem as the following optimal control problem.

$$\min_{u^F} J_F = f^{F^T} L^T L f^F + \gamma f^{F^T} f^F \quad (8)$$

$$s.t. \quad F_e = Q f^F \quad (9)$$

$$\underline{c} \leq f^F \leq \bar{c} \quad (10)$$

where $L = [[r_1], [r_2], [r_3], [r_4]]$ and $[r]$ is the cross-product matrix. γ is the regulatory factor to adjust the uniform distribution of foot force. The first item of objective function aiming resultant the whole moment as zero as possible, and the second item for reducing effort. $Q = [1_3, 1_3, 1_3, 1_3]$. $\underline{c}$ and $\bar{c}$ are the lower and upper bounds of force. By solving a quadratic convex programming problem, we obtain additional foot force f^F . So as the torque compensator generate f^T for rotational motion.

At last, based on the consideration of the limited joint torque and friction constrain, a clipper makes the desired ground reaction force f^C to meet the physical feasibility.

3. Reinforcement Learning for PDMPC

Controllers with fixed parameters make it difficult to adapt the robot to different states of motion. For example, the gait cycle of the robot will be different when it moves at diverse speeds, so the parameters need to change adaptively. Manual adjustment for the control framework with large-scale parameters is laborious and time-consuming, and the results of parameter adjustment are sometimes tricky to achieve the intended goals.

In this section, the reinforcement learning method is used to establish the relationship between the robot states and the controller parameters, so that the multiple parameters can be automatically adjusted. Figure 4 shows the framework of reinforcement learning based on PDMPC.

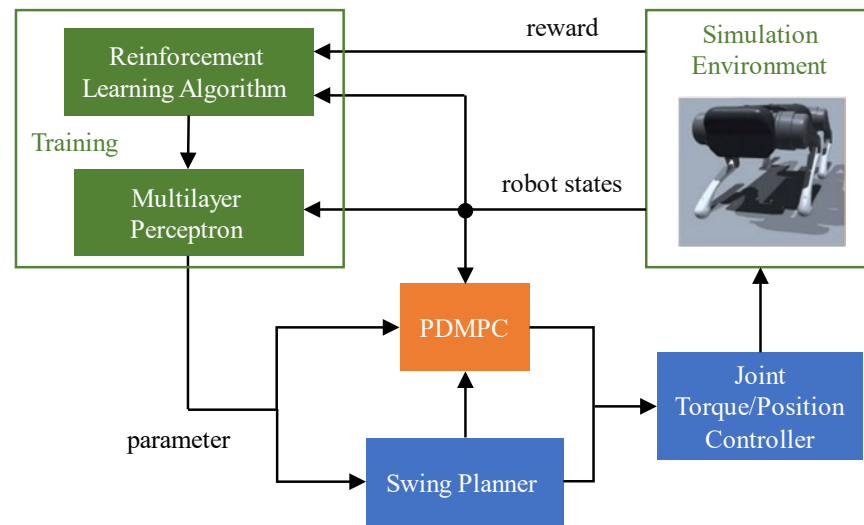


Figure 4. Framework of reinforcement learning based on PDMPC.

We utilize the open-source PPO algorithm [16] to train the policy according to the states and the reward provided by the simulation environment. The action policy network is an MLP neural network that receives the current robot states and outputs the parameters for the PDMPC controller and the swing planner. According to the current parameters, the swing planner determines the gait frequency and the target position of the swing leg, the PDMPC controller determines the desired ground reaction force for the stance leg. Finally, the joint controller performs joint control.

3.1. Parameters to Be Larned

3.1.1. Swing Planner

The swing planner is used to choose the gait pattern, and determines the phase relation and lift-off schedule. The duty cycle is 0.5, and the result schedule information will be transmitted to the PDMPC controller. The foot point position vector ${}^w\mathbf{h}$ in the world coordinate system is as follows.

$$\begin{cases} {}^w\mathbf{h}_{xy} = {}^w\mathbf{h}_{0xy} + (\alpha_x \mathbf{V}_x + \alpha_y \mathbf{V}_y) t_s \\ {}^w\mathbf{h}_z = {}^w\mathbf{h}_{0z} + \mathbf{1}(\varphi) l_z \end{cases} \quad (11)$$

where $\mathbf{h}_0$ is the initial foot point position vector at the lift-off event. The foot trajectory is determined by the heuristic of single inverted pendulum model, t_s is the duration of the support period, α_x and α_y is the heuristic coefficient in different motion directions (normally set to 0.5). l_z is the maximum height of foot in vertical direction during swing

phase. The gait phase variable $\varphi \in [0, 2\pi]$, $\mathbf{1}(\varphi)$ is an indicator function, 1 stands for swing period, 0 stands for support period.

$$\mathbf{1}(\varphi) = \begin{cases} 0 & \varphi \in (0, \pi] \\ 1 & \varphi \in (\pi, 2\pi] \end{cases} \quad (12)$$

So, we have 7 parameters to be learned for swing planner, including the four lift heights of each leg l_{z1} , l_{z2} , l_{z3} and l_{z4} , the two heuristic coefficients α_x and α_y , the half support duration $t_s/2$.

3.1.2. PDMPC Controller

The PDMPC controller solves the required ground reaction force according to the robot states and the schedule information provided by the swing planner. In our previous work, the manual parameter adjustment takes a long time and has no adaptability. Therefore, it is helpful to improve control performance by incorporating these parameters into the learning process.

There are nine parameters to be learned for PDMPC, including the vertical force coefficients k_{fz} and d_{fz} , the horizontal velocity coefficients d_{fx} and d_{fy} , the roll and pitch torque coefficients $k_{\tau\phi}$, $k_{\tau\theta}$, $d_{\tau\phi}$ and $d_{\tau\theta}$, the yaw angular velocity coefficient $d_{\tau\psi}$.

For the low-level joint torque/position controller, the PD gain for position tracking is easy to turn, therefore, it is unnecessary to put it into our learning process.

3.2. Policy Network

The action policy obeys a multidimensional normal distribution $\pi \sim N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, The mean vector is $\boldsymbol{\mu}$, and the covariance matrix is $\boldsymbol{\Sigma}$. The covariance matrix is used for exploration, and its elements can be gradually reduced with time to make the training converge. $\boldsymbol{\mu} = f(\boldsymbol{\theta})$, f consists of a full connect neural network, $\boldsymbol{\theta}$ are the network parameters.

Figure 5 shows the schematic diagram of the neural network structure of the action policy, including three hidden layers, the orange balls, the number of units is 256, 128 and 64, respectively. Batch regularization processing is used for inputs [17], and exponential linear units (ELU) are used as activation function.

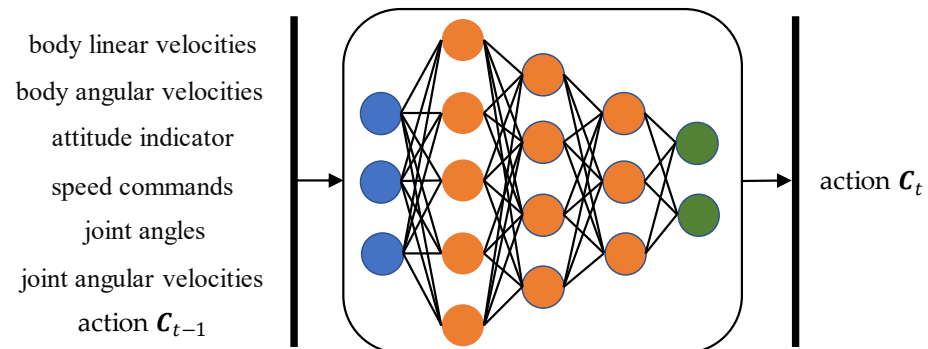


Figure 5. The structure of action policy neural network.

The input is a 52-dimensional vector, including a 3-dimensional robot body linear velocity vector, a 3-dimensional body angular velocity vector, a 3-dimensional attitude indicator vector, a 3-dimensional speed command vector, a 12-dimensional joint position vector, a 12-dimensional joint angular velocity vector and a 16-dimensional network output action vector at the previous step C_{t-1} . The attitude indicator vector refers to the projection of the unit vector in the gravity direction under the body coordinate system. The speed command includes two linear speeds in the horizontal direction and the yaw angular velocity. C_t is not used directly, its elements are converted to the required parameters for swing planner and PDMPC controller through appropriate mapping, as shown in Table 1.

Table 1. The transform between the network action and the parameters for PDMPC.

Parameter	Transform	Parameter	Transform
l_{z1}	$0.03 + c_1/100$	d_{fy}	$50 + c_9 \cdot 10$
l_{z2}	$0.03 + c_2/100$	k_{fz}	$200 + c_{10} \cdot 50$
l_{z3}	$0.03 + c_3/100$	d_{fz}	$20 + c_{11} \cdot 5$
l_{z4}	$0.03 + c_4/100$	$k_{\tau\varphi}$	$50 + c_{12} \cdot 10$
$t_s/2$	$200 + c_5 \cdot 10$	$d_{\tau\varphi}$	$8 + c_{13} \cdot 2$
α_x	$0.5 + c_6/10$	$k_{\tau\theta}$	$50 + c_{14} \cdot 10$
α_y	$0.5 + c_7/10$	$d_{\tau\theta}$	$8 + c_{15} \cdot 2$
d_{fx}	$50 + c_8 \cdot 10$	$d_{\tau\psi}$	$(1 + c_{16}) \cdot 5$

The pseudo code of reinforcement learning based on PDMPC is as follow (Algorithm 1).

Algorithm 1 Reinforcement learning based on PDMPC

Input θ_0 initial parameter of action network
 x_0 initial parameter of state value function V
for $k = 0, 1, 2, \dots$ **do**
 sample parameter vector C from π_{θ_k}
 $C_{gait} \rightarrow$ swing planner, assign leg states $1(\varphi)$ and target positions ${}^w h$
 $C_{mpc} \rightarrow$ PDMPC, desired ground reaction force f^F
 joint Controll $\rightarrow \tau$
 sample trajectory $\Phi_k = \{\tau_i\}_{i=1,2,\dots,n}$
 if reset_flag **then**
 reset robot
 end if
 if data sufficient **then**
 compute reward $\hat{R}_t$
 estimate advantage function $\hat{A}_t$ based on state value function V_{x_k}
 compute Clipped Surrogate Objective (PPO) M
 update policy by gradient ascent algorithm (G):

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{|\Phi_k|T} \sum_{T \in \Phi_k} \sum_{t=0}^T M$$
 fitting V_{x_k} by quadratic mean square regression, update parameter:

$$x_{k+1} = \arg \min_x \frac{1}{|\Phi_k|T} \sum_{T \in \Phi_k} \sum_{t=0}^T (V_{x_k}(s_t) - \hat{R}_t)^2$$
 end if
end for

4. Simulation and Result

4.1. Simulation Platform

We constructed a quadruped robot control algorithm software platform, and its architecture is shown in Figure 6. The platform can be divided into three layers according to functions, namely control architecture layer, conversion layer and training layer.

The control layer runs the traditional manually designed controllers, the code is written in C++ and integrates many open-source mathematical libraries (GSL, Eigen, and qpOASES), which improve the efficiency of algorithm development. The algorithm code has good transplant characteristics and can be rapidly deployed to different dynamic environments, such as Gazebo and other robot simulation environments, as well as real quadruped robots.

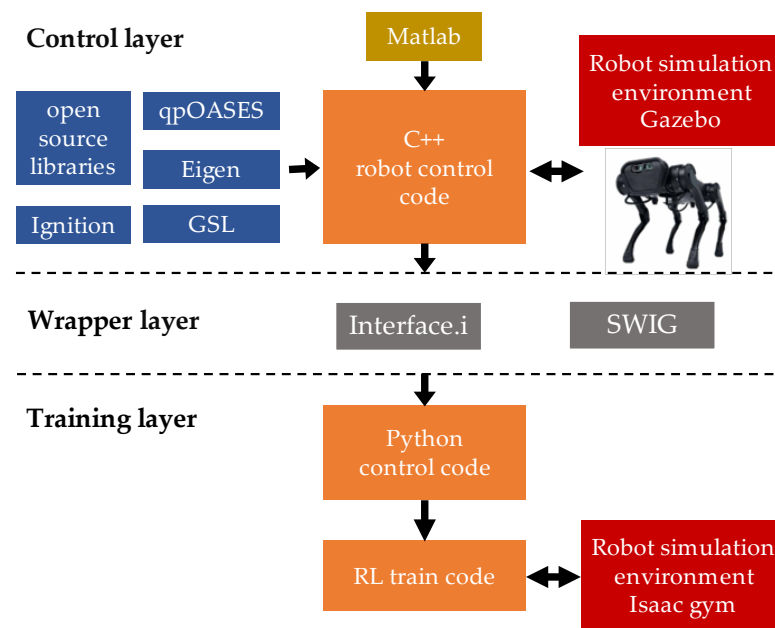


Figure 6. The software architecture of quadruped robot control.

The wrapper layer converts the C++ code of the control layer to other types of programming languages to meet the needs of different environments. SWIG-4.0.2 (Simplified Wrapper and Interface Generator) is a software development tool used to build C and C++ program script language interfaces. It can quickly package C/C++ code into Python, Perl, Ruby, Java and other languages. The reinforcement learning we used requires Python language. Declare the C++ control algorithm function in the interface file *Interface.i* as required, and then convert it by SWIG.

The training layer consists of the converted Python control code library, the learning and training algorithm and Isaac Gym, where Isaac Gym is a physical simulation environment specially developed for reinforcement learning research [18].

4.2. Training and Rewards

The simulation was conducted on a desktop laptop with eight CPU cores (Intel Core I7-7700HQ) and single GPU (NVIDIA GeForce GTX 1070). In Isaac Gym environment, we train 20 quadruped robots in parallel. The robot uses a diagonal trotting gait, the simulation time step is 0.005 s, the control frequency of PDMPC is 100 Hz, the maximum alive duration is 15 s, the linear speed command range is [0, 1] m/s, and the angular speed command range is [−1, 1] rad/s.

Algorithm 1 and Adam optimizer [19] are used to train the policy network, and the corresponding hyper-parameters are listed in Table 2.

Table 2. Hyper-parameters in Algorithm 1.

Hyper-Parameters	Value
decay factor	0.99
clip factor	0.2
KL divergence threshold	0.0008
learning rate	$3e - 4$
batch size	960

The reward function we designed is as follows:

- Task target reward

For the task of locomotion with the command speed, we encourage the robot with smaller speed error respect to expectation. The nonlinear exponential function makes the

robot obtain much more score when the speed tracking performance improved a little, especially when the robot have medium tracking ability.

$$r_{rew} = \beta_{lvxy} \exp\left(-3\|{}^B\mathbf{V}_{xy} - \mathbf{V}_{cxy}\|_2\right) + \beta_{a\omega} \exp\left(-3\left({}^B\boldsymbol{\omega} - \boldsymbol{\omega}_c\right)^2\right) \quad (13)$$

- Balance punishment

On the other hand, we punish the robot with Equation (14).

$$r_{pun} = -\beta_{lvz}v_z^2 - \beta_{a\varphi\theta}\left(\varphi^2 + \theta^2\right) - \beta_g\left\|\left({}^B\mathbf{T}\mathbf{e}_g\right)_{xy}\right\|_2 \quad (14)$$

where ${}^B\mathbf{T}$ is the rotation matrix from the world coordinate system to the body coordinate system, and $\mathbf{e}_g$ is the unit vector in gravity direction. The first item punishing the robot is unable to maintain the body height stable; the remaining item punishes the robot with unnecessary roll and pitch movement.

All β coefficients adjust each item to form the whole reward to evaluate the current policy. Figure 7 shows that the training reward gradually increases with the increase of the number of samples, and the number of samples required is 10^6 .

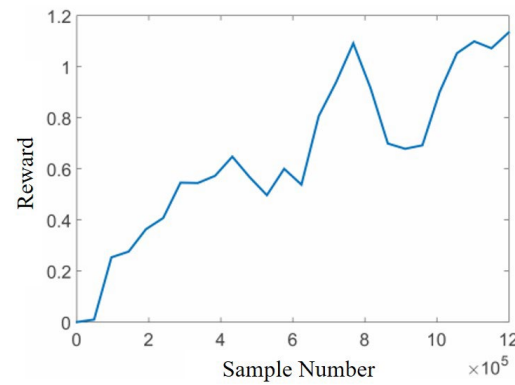


Figure 7. Training reward.

4.3. Result and Discussion

We first compare the locomotion performance of the fixed parameter PDMPC and the learned PDMPC. Figure 8 shows the velocity tracking performance of each controller. Obviously, the robot under the learned controller has smaller tracking error, and the motion is more smooth.

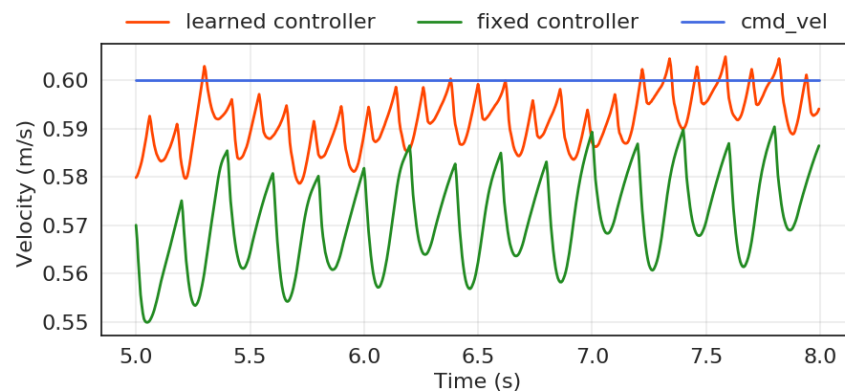


Figure 8. Forward velocity tracking performance.

Next, we check the gait and stability at different speeds. The forward velocity command gradually increases from 0 to 1 m/s. As shown in Figures 9 and 10, when the

command speed increases, the fixed parameter controller holds the same gait, but the learned controller reduces its phase time. This change is in line with the biological norm that animals have a higher frequency of gait at high speeds. In addition, the attitude angle of the robot under the learned controller is closer to 0, and its oscillation amplitude is smaller. This phenomenon indicates that the learned controller is more capable of absorbing the impact of the swing leg when it touches down.

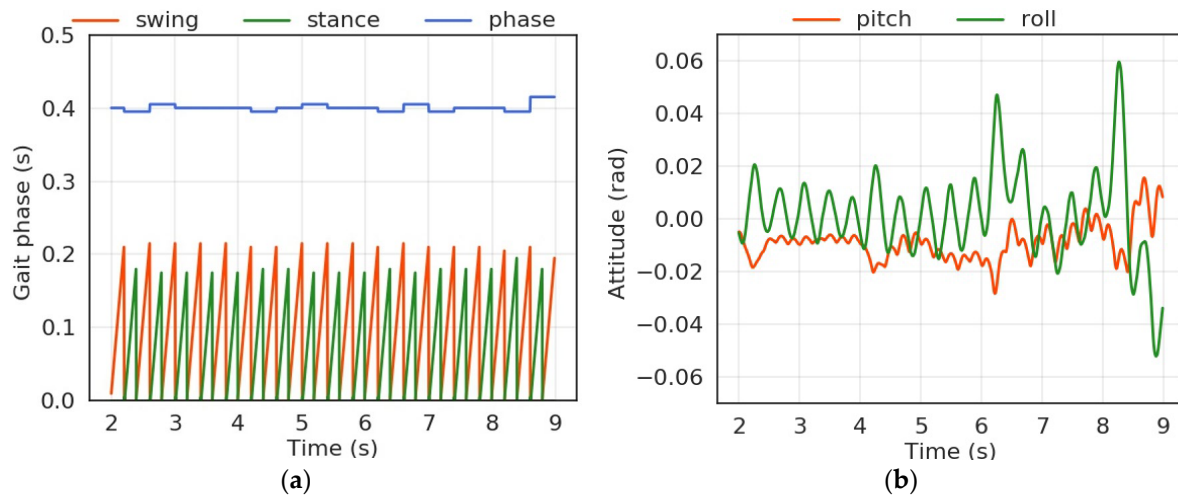


Figure 9. Motion controlled by fixed parameter PDMPC. (a) Gait; (b) Attitude angle.

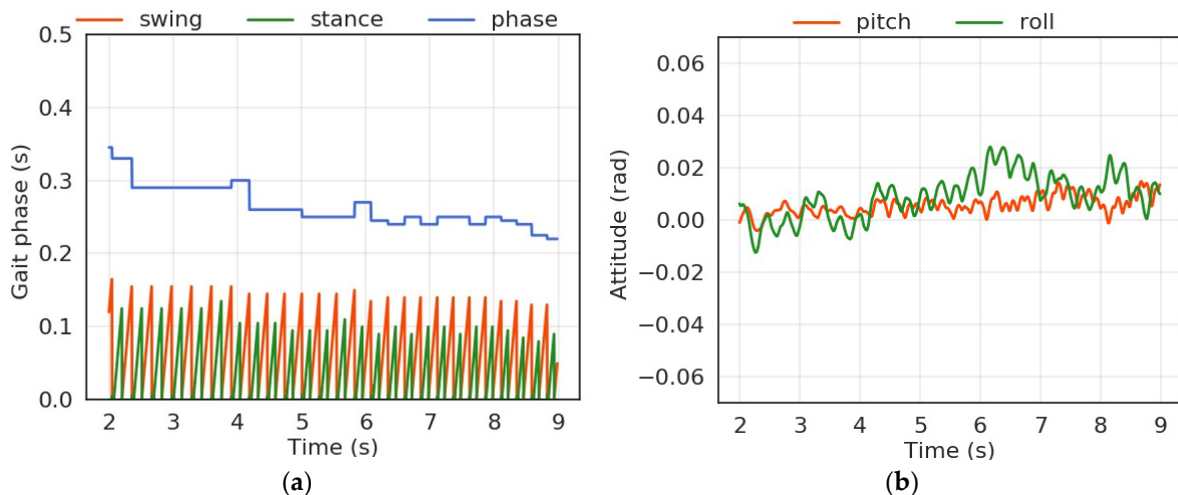


Figure 10. Motion controlled by learned parameter PDMPC. (a) Gait; (b) Attitude angle.

Figure 11 shows the adaptive changes of parameters of PDMPC during the acceleration of the quadruped robot. Figure 11a shows the adaptive heuristic coefficients obtained through reinforcement learning. The forward and lateral coefficients increase with the increase of speed. Manual-designed swing trajectory usually sets the heuristic coefficient of the inverted pendulum heuristic to a fixed value of 0.5. Because the model cannot be strictly regarded as a single-stage inverted pendulum during the motion, it is not reasonable to use fixed coefficients.

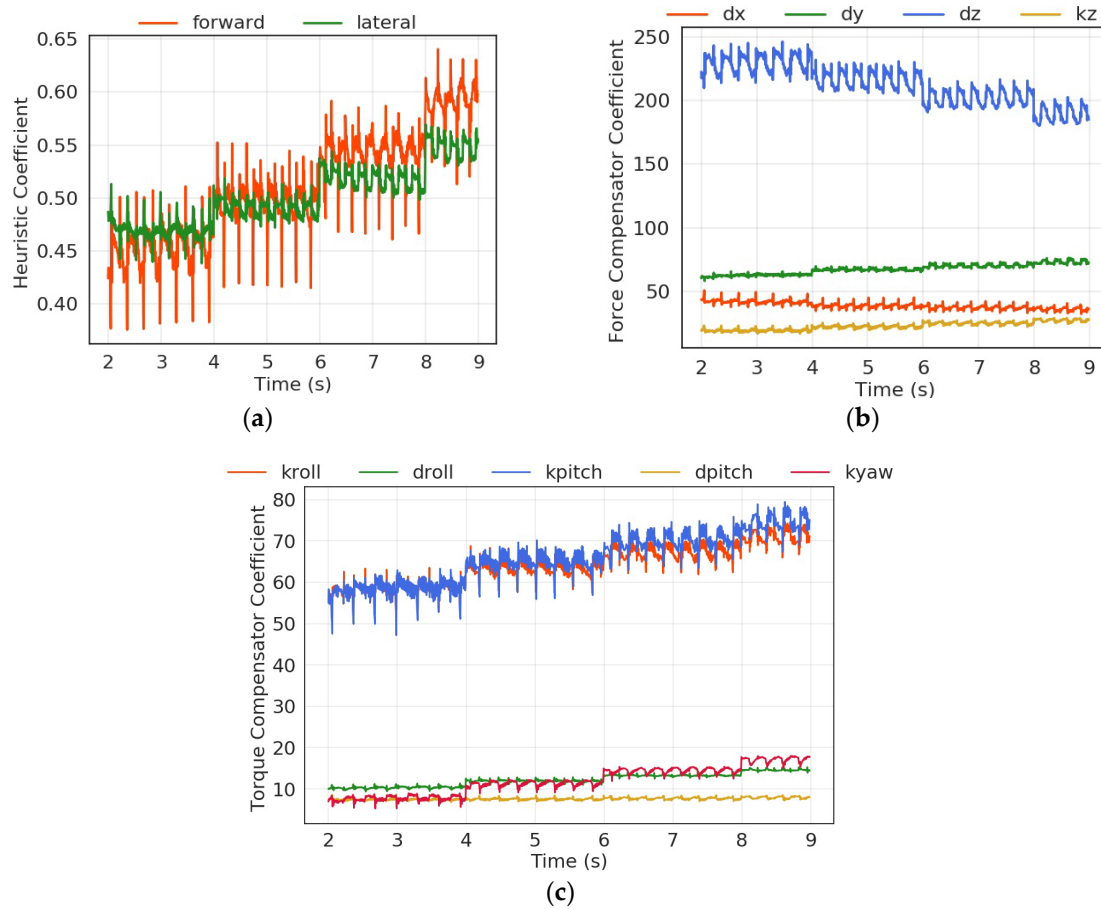


Figure 11. The adaptive variation of parameters during the acceleration of the quadruped robot. (a) Parameters for swing planner; (b) Parameters for force compensator; (c) Parameters for torque compensator.

It can be seen in Figure 11b that when the robot speed increases, the decreasing parameter d_x makes the forward speed tracking of the robot softer and the increasing dy strengthens the lateral control at the same time; the decreasing parameter k_z and d_z reduce the position compensation in the height direction of the robot and increase the damping, making the position of the center of mass more flexible. At the same time, the parameters of the torque compensator are all increased, which is conducive to making the robot's attitude more stable during acceleration.

5. Conclusions and Future Work

In this work, we proposed a novel locomotion control algorithm for a quadruped robot which combines the advantages of model-based MPC and model-free reinforcement learning. PDMPC controller performs a fundamental locomotion capability that provides a safe exploration region, and reinforcement learning endows robots with evolutionary ability. The trained policy chooses the parameters for the PDMPC controller adaptively according to the current state. The simulation results show the effectiveness of our algorithm, compared with the fixed parameters controller, the adaptive parameters make the learned controller have better performance in command tracking and equilibrium stability, and the gait of robot changes with speed just as quadrupeds do.

Model-free RL learns an end-to-end control policy based on the reward function to maximize performance. This learning from scratch requires massive samples, which means a large number of robots and time consumption. In this work, with the model-based controller to generate good samples, our training process has been greatly shortened. It requires fewer robots and has a faster learning speed. On the other hand, the manually

designed controller will guide the policy into a local optimal region, limiting the powerful exploration, discovery and learning ability of reinforcement learning. As two major frameworks for solving optimal control problems, conventional control and learning-based control have their own advantages and can promote and develop together. We will explore more ways to merge them and make online quick learning possible on the physical platform.

Author Contributions: Z.Z. and X.C. designed this research methods and wrote the manuscript; H.M., H.A. and L.L. edited and revised the manuscript. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by National Science Foundation of China (No. 61903131) and the China Postdoctoral Science Foundation (No. 2020M683715).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Not applicable.

Conflicts of Interest: The authors declare no conflict of interest.

References

- Kimura, H.; Fukuoka, Y.; Hada, Y.; Takase, K. Three-dimensional Adaptive Dynamic Walking of a Quadruped rolling motion feedback to CPGs controlling pitching motion. In Proceedings of the 2002 IEEE International Conference on Robotics and Automation, Washington, DC, USA, 11–15 May 2002.
- Farshidian, F.; Neunert, M.; Winkler, A.W.; Rey, G.; Buchli, J. An efficient optimal planning and control framework for quadrupedal locomotion. In Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), Marina Bay Sands, Singapore, 29 May–3 June 2017; pp. 93–100.
- Neunert, M.; Farshidian, F.; Winkler, A.W.; Buchli, J. Trajectory Optimization Through Contacts and Automatic Gait Discovery for Quadrupeds. *IEEE Robot. Autom. Lett.* **2017**, *2*, 1502–1509. [\[CrossRef\]](#)
- Richalet, J.; Rault, A.; Testud, J.; Papon, J. Model predictive heuristic control: Applications to an industrial process. *Automatica* **1978**, *14*, 413–428. [\[CrossRef\]](#)
- Carlo, J.D.; Wensing, P.M.; Katz, B.; Bledt, G.; Kim, S. Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control. In Proceedings of the 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Madrid, Spain, 1–8 October 2018; pp. 1–9.
- Bledt, G.; Wensing, P.M.; Kim, S. Policy-regularized model predictive control to stabilize diverse quadrupedal gaits for the MIT cheetah. In Proceedings of the 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Vancouver, BC, Canada, 24–28 September 2017; pp. 4102–4109.
- Ding, Y.; Pandala, A.; Li, C.; Shin, Y.-H.; Park, H.-W. Representation-Free Model Predictive Control for Dynamic Motions in Quadrupeds. *IEEE Trans. Robot.* **2021**, *37*, 1154–1171. [\[CrossRef\]](#)
- Liu, X.; Ma, H.; An, H.; Wei, Q.; Lang, L. Gait Parameters Design Method of Trotting Gait Based on Energy Consumption. In Proceedings of the 2021 6th IEEE International Conference on Advanced Robotics and Mechatronics (ICARM), Chongqing, China, 3–5 July 2021; pp. 174–179.
- Chang, X.; Ma, H.; An, H. Quadruped Robot Control through Model Predictive Control with PD Compensator. *Int. J. Control. Autom. Syst.* **2021**, *19*, 3776–3784. [\[CrossRef\]](#)
- Kolter, J.Z.; Ng, A.Y. Policy search via the signed derivative. In *Robotics: Science and Systems*; The MIT Press: Cambridge, MA, USA, 2009.
- Haarnoja, T.; Zhou, A.; Ha, S.; Tan, J.; Tucker, G.; Levine, S. Learning to Walk via Deep Reinforcement Learning. *arXiv* **2018**, arXiv:1812.11103.
- Yao, Q.; Wang, J.; Yang, S.; Wang, C.; Zhang, H.Y.; Zhang, Q.F.; Wang, D.L. Imitation and Adaptation Based on Consistency: A Quadruped Robot Imitates Animals from Videos Using Deep Reinforcement Learning. *arXiv* **2022**, arXiv:2203.05973.
- Hwangbo, J.; Lee, J.; Dosovitskiy, A.; Bellicoso, D.; Tsounis, V.; Koltun, V.; Hutter, M. Learning agile and dynamic motor skills for legged robots. *Sci. Robot.* **2019**, *4*, eaau5872. [\[CrossRef\]](#) [\[PubMed\]](#)
- Lee, J.; Hwangbo, J.; Wellhausen, L.; Koltun, V.; Hutter, M. Learning quadrupedal locomotion over challenging terrain. *Sci. Robot.* **2020**, *5*, eabc5986. [\[CrossRef\]](#) [\[PubMed\]](#)
- Kober, J.; Bagnell, J.A.; Peters, J. Reinforcement learning in robotics: A survey. *Int. J. Robot. Res.* **2013**, *32*, 1238–1274. [\[CrossRef\]](#)
- Schulman, J.; Wolski, F.; Dhariwal, P. Proximal Policy Optimization Algorithms. *arXiv* **2017**, arXiv:1707.06347.
- Ioffe, S.; Szegedy, C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In Proceedings of the International Conference on Machine Learning (ICML), Lille, France, 6–11 July 2015.

18. Rudin, N.; Hoeller, D.; Reist, P.; Hutter, M. Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning. In Proceedings of the 5th Annual Conference on Robot Learning, London, UK, 8 November 2021.
19. Kingma, D.P.; Ba, J. Adam: A Method for Stochastic Optimization. In Proceedings of the 3rd International Conference for Learning Representations, San Diego, CA, USA, 7–9 May 2015.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.